MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Faculty Name	Sanjeev Khatri		
Subject Name with Code	4CS4-06 Introduction to Java Programming		
Semester	4th	**Session**	2023-24 (Even)
Branch	CSE	**Batch**	A
Unit No.	1	**Submission Date**	08-Feb-24

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Object Oriented Programming Principles	1 - 4	4
2	Java Essentials, Java Virtual Machine	5 - 6	2
3	Program Structure in Java, Java Class Libraries	7 - 8	2
4	Data Types, Variables	9 - 12	4
5	Casting, Automatic Type Promotion in Expressions	13 - 14	2
6	Operators, The ? Operator, Operator Precedence	15 -17	3
7	Arrays	18 - 20	3
8	Control Statements	21 - 26	6

Faculty

HOD

13/2/24

Dean Academics

UNIT-1

LECTURE - 1

Object Oriented Programming Principles

What is Java ?

Java is an object-oriented programming language developed by Sun Microsystems, a company best known for its high-end Unix workstations. Modeled after C++, the Java language was designed to be small, simple, and portable across platforms and operating systems, both at the source and at the binary level. it allows computer programmers to implement an object-oriented design as a working system languages like C++.

Programming Paradigm -

Programming paradigms are the ways to classify programming languages based on their features. Not only programming languages but all fields like psychology, sociology and others use paradigms because every field have its own development age and every development has various different features.

Common Programming Paradigm-

- **Imperative** - In computer science, imperative programming is a programming paradigm of software that uses statements that change a program's state. In much the same way that the imperative mood in natural languages expresses commands, an imperative program consists of commands for the computer.
- **Declarative** - in which the programmer merely declares properties of the desired result, but not how to compute it.
- **Structured** - in which programmers works with clean, nested control structures.
- **Procedural** - It has been derived from imperative programming, based on the concept of the procedure call. Procedures simply contain a series of computational steps to be carried out. Any given procedure might be called at any point during a program's execution.
- **Functional** - It is a programming paradigm where programs are constructed by applying and composing functions. It is a declarative programming paradigm in which function definitions are trees of expressions that map values to other values, rather than a sequence.

MITRC

- **Object Oriented** – It is based on the concept of "objects", which can contain data and code: data in the form of fields, and code, in the form of procedures.

Characteristics of Object-Oriented Programming Languages-

Objects

Object is an instance of class. It is a piece of code which represents the real-life entity. When we define a class, no memory is allocated but when we create an object, memory is allocated. Each object contains data and code to manipulate the data. Without class object doesn't exists.

Example - Student

When we discuss about any student, we need his name, date of birth, class, marks, etc. which called Data.

After that we need to calculate total of marks, average marks, we need to create various functions like Total, Average, Output, etc. for reusability.

Class

Class is a collection of objects. It serves a blueprint of data and functions available in any object. It is a user-defined data type, which holds its own data members and member functions. Class is a structure where we can define variables, methods to utilize them by an object. Without object we can use static functions of class.

Example -

```
class Student
{
        name;
        dateofbirth;
        classname;
        marks1;
        marks2;
        marks3;
        total();
        average();
```

MITRC

```
        display();
}
```



Now we create object like - Student st;

Encapsulation

When we wrap up data and information under a single unit, it is known as encapsulation. It hides data for purpose of protection.

Example - College

It is an example of encapsulation where we have various departments like accounts, labs, student management, etc.

Data Abstraction

Abstraction is nothing but it hides implementation details while just presenting the features to the outside world. It refers to the act of representing essential features of any thing without including the background details or explanation. It reduce code complexity.

Example-

A real-life example is out Mobile Phone. We turn it on, make calls, click pictures, etc. but we do not know how all these thing works. How a phone gets connect with operators and other person, how it converts any click to a picture, etc.

Similarly, we call a function of a class in a program without knowing how it is working.

Polymorphism

Poly means many and morphism means forms means many forms. In other words, polymorphism is a Greek term that means the ability to take more than one form. Or we can say the ability of a message to be displayed in more than one form.

If something possesses different behaviour in different situations, this is known as polymorphism. Like a person can be father, husband, son, teacher at the same time.

Dynamic Binding

MITRC

Binding refers to the linking of a procedure call to the code to the executed in response to the call. Dynamic binding means the code associated with a given procedure call is not known until the time of the call at run-time. It is associated with a polymorphic reference depends upon the dynamic type of that reference.

Message Passing

An Object-Oriented program consists of a set of objects that communicate with each other.

A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

Example-

Student ⟵ Object

MarksAverage (Marks1, Marks2, Marks3)

Message Information

Inheritance

An Object-Oriented program consists of a set of objects that communicate with each other.

A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

It is a mechanism of deriving a new class from an existing class.

- ❑ Single level Inheritance - A derived class is created from single base class like rose class derived from flower class (base class).
- ❑ Multi-Level Inheritance - A derived class is created from another derived classes and the same base class of another derived classes like World-India-Rajasthan-Alwar.
- ❑ Multiple Inheritance - A derived class is created from another derived class like Bird may be a parrot or a sparrow.
- ❑ Hierarchical Inheritance - A derived class is created from more than one base class like World-India-Delhi or Rajasthan and USA-New York or California.
- ❑ Hybrid Inheritance - This is combination of more than one inheritance like hierarchical + multiple or multi-level + multiple.

MITRC

Benefits of Object-Oriented Programming Languages-

- ❑ Allows programmers to Develop Softwares.
- ❑ Through Inheritance, we can eliminate redundant code and extend the use of existing classes.
- ❑ Through Objects, we can easily partition our work in a project.
- ❑ It is possible to have multiple instances of an object to co-exist without any interference.
- ❑ Software complexity can be easily managed.
- ❑ Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- ❑ It overcome flaws of the procedural approach to programming such as Reusability, Maintainability.
- ❑ It can combine Data and Functions in a single unit.
- ❑ It can easily upgrade from small system to large system.
- ❑ It is simple to implement.

Advantages of Learning Java

- ➢ **Java Is Platform-Independent** - Platform independence is one of the most significant advantages that Java has over other programming languages, particularly for systems that need to work on many different platforms. Platform-independence is a program's capability of moving easily from one computer system to another.
- ➢ **Java Is Object-Oriented** - Java's object-oriented concepts are inherited from C++, the language on which it is based, but it borrows many concepts from other object-oriented languages as well. Like most object-oriented programming languages, Java includes a set of class libraries that provide basic data types, system input and output capabilities, and other utility functions.
- ➢ **Java Is Easy to Learn** - Java's initial design goals was to be small and simple, and therefore easier to write, easier to compile, easier to debug, and, best of all, easy to learn.
- ➢ Java was developed to be architecturally neutral, which means that anyone can build an application without extensive study.
- ➢ After we write a Java program, the compiler converts the source code into a binary program of bytecode.
- ➢ Java programs that are embedded in a Web page are called applets, while standalone programs are called Java applications.

MITRC

LECTURE - 2

Java Essentials, Java Virtual Machine

Java can be run on a wide variety of computers and devices because it does not execute instructions on a computer directly. Instead, Java runs on a hypothetical computer known as the Java Virtual Machine (JVM). When programmers call the JVM hypothetical, they mean it is not a physical entity created from hardware, but is composed only of software.

To write a program in Java, we need the Java Development Kit (JDK). Strictly speaking, the JDK is all we need; however, other tools can make writing a Java application easier. Most software developers like to use an ***Integrated Development Environment (IDE)*** such as NetBeans and Eclipse. An IDE can be very helpful when developing and deploying commercial applications.

JDK

JDK is a collection of programs that enables us to write programs in Java. The two programs we'll use most are javac.exe and java.exe. The javac.exe program is the compiler, which means it's the program that turns code we can read (the code you write in Java) into code our computer can read (the collection of 0s and 1s that a computer needs when it runs a program). The java.exe program runs the programs that we write. The JDK is platform dependent. It means that for every different platform, you require a different JDK. it consists of various tools for debugging, monitoring, and developing java applications.

What is JRE?

JRE stands for Java Runtime Environment- also written as Java RTE. It is a set of software tools designed for running other software. It is an implementation of JVM, and JRE provides a runtime environment. JRE, just like JDK, is also platform dependent. It means that for every different platform, we require a different JRE. It does not consist of any tool- like a debugger, compiler, etc.

Java Virtual Machine (JVM)

The Java virtual machine manages application memory and provides a portable execution environment for Java-based applications. Developers reap the rewards in performance, stability, and predictable runtimes. The Java virtual machine is a program whose purpose is

MITRC

to execute other programs. The JVM is platform independent. It means that we won't require a different JVM for every different platform. JVM does not consist of any tools for software development.

Why use JDK?

- JDK contains tools required to write Java programs, and JRE to execute them.
- It includes a compiler, Java application launcher, Appletviewer, etc.
- Compiler converts code written in Java into byte code.
- Java application launcher opens a JRE, loads the necessary class, and executes its main method.

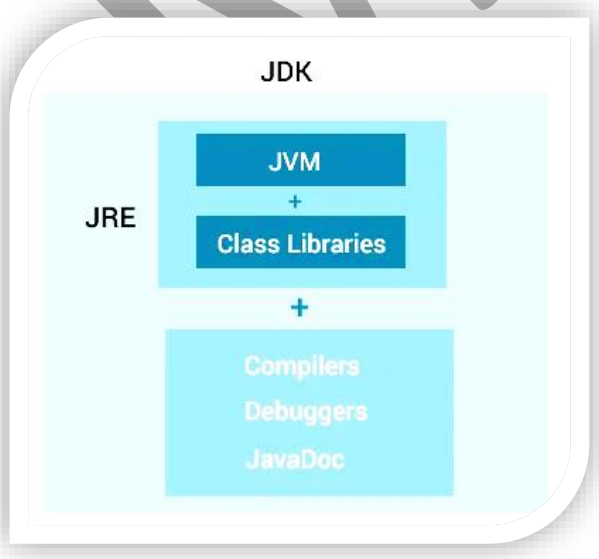
Why use JRE?

- JRE contains class libraries, JVM, and other supporting files. It does not contain any tool for Java development like a debugger, compiler, etc.
- It uses important package classes like math, swing, util, lang, awt, and runtime libraries.
- If you have to run Java applets, then JRE must be installed in your system.

Why JVM?

- JVM provides a platform-independent way of executing Java source code.
- It has numerous libraries, tools, and frameworks.
- Once you run Java program, you can run on any platform and save lots of time.
- JVM comes with JIT(Just-in-Time) compiler that converts Java source code into low-level machine language. Hence, it runs more faster as a regular application.

Relationship between JVM, JRE, and JDK.



MITRC

LECTURE - 3

Program structure in java, Java class libraries

Java can be run on a wide variety of computers and devices because it does not execute instructions on a computer directly. Instead, Java runs on a hypothetical computer known as the Java Virtual Machine (JVM). When programmers call the JVM hypothetical, they mean it is not a physical entity created from hardware but is composed only of software.

When we write a Java program, we first construct the source code using a text editor such as Notepad, VS Code or a development environment and source code editor such as jGRASP, which we can download from the Web for free. A development environment is a set of tools that help us write programs by providing such features as displaying a language's keywords in color. The statements are saved in a file; then, the Java compiler converts the source code into a binary program of bytecode. A program called the Java interpreter then checks the bytecode and communicates with the operating system, executing the bytecode instructions line by line within the Java Virtual Machine. The JVM provides security against intruders accessing our computer's hardware through the operating system. Therefore, Java is more secure than other languages.

Java Program Types

We can write two kinds of programs using Java:

1. Applets are programs that are embedded in a Web page.
2. Java applications are stand-alone programs. Java applications can be further subdivided into console applications, which support character or text output to a computer screen, and windowed applications, which create a GUI with elements such as menus, toolbars, and dialog boxes. Console applications are the easier applications to create.

Java Program Structure-

```
import java.util.*;

public class hello

{

        public static void main(String[] args)
```


MITRC

```
    {
        System.out.println("Hello World");
    }
}
```

In above program-

1. import java.util.*; ← we included pre-defined libraries. (Modern text editors we indirectly supports these libraries and can write our program without use of common libraries.)
2. public class hello {} ← The gateway of a java program where we write our main content which will execute at the time of start of a program. This line is the class header.
3. System.out.println("Hello World"); ← This is the class body which we write between the curly braces.

Java Class Library-

The Java Class Library (JCL) is a set of dynamically loadable libraries that Java Virtual Machine (JVM) languages can call at run time. Because the Java Platform is not dependent on a specific operating system, applications cannot rely on any of the platform-native libraries. Instead, the Java Platform provides a comprehensive set of standard class libraries, containing the functions common to modern operating systems.

Comments in Java-

Line comments start with two forward slashes (//) and continue to the end of the current line. A line comment can appear on a line by itself or at the end (and to the right) of a line following executable code. Line comments do not require an ending symbol.

Block comments start with a forward slash and an asterisk (/*) and end with an asterisk and a forward slash (*/). A block comment can appear on a line by itself, on a line before executable code, or on a line after executable code. Block comments also can extend across as many lines as needed.

Javadoc comments are a special case of block comments called documentation comments because they are used to automatically generate nicely formatted program documentation with a program named javadoc. Javadoc comments begin with a forward slash and two asterisks (/**) and end with an asterisk and a forward slash (*/).

MITRC

LECTURE - 4

Data types, Variables

At the foundation of any programming language are its data types and operators, and Java is no exception. These elements define the limits of a language and determine the kind of tasks to which it can be applied. Data types are especially important in Java because it is a strongly typed language. In core Java, there are eight primitive (also called elemental or simple) types of data -

Type	Meaning
boolean	Represents true/false values
byte	8-bit integer
char	Character
double	Double-precision floating point
float	Single-precision floating point
int	Integer
long	Long integer
short	Short integer

Integers- Java defines four integer types: byte, short, int, and long, which are shown here:

Type	Width in Bits	Range
byte	8	–128 to 127
short	16	–32,768 to 32,767
int	32	–2,147,483,648 to 2,147,483,647
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

```
public class sumoftwovalues

{
	public static void main(String[] args)

	{
		int a = 5;
		int b = 10;
		int c = a + b;
		System.out.println("Sum of two values : " + c);
	}
}
```

Output-

Sum of two values : 15

Floating-Point - There are two kinds of floating-point types, float and double.

```
public class sumoftwovalues {

    public static void main(String[] args)
    {
        double a = 5.6;
        double b = 11.5;
        double c = a + b;
        System.out.println("Sum of two values : " + c);
    }
}
```

Output-

Sum of two values : 17.1

Characters - In Java, characters are not 8bit quantities like they are in many other computer languages. Instead, Java uses Unicode. Unicode defines a character set that can represent all the characters found in all human languages. In Java, char is an unsigned 16bit type having a range of 0 to 65,535.

```
class chardemo {

    public static void main(String[] args)
    {
        char ch;
        ch = 'X';
        System.out.println("Ch holds : " + ch);
        ch++;
        System.out.println("New Value of Ch : " + ch );
        ch = 65;
        System.out.println("New Value of Ch : " + ch );
    }

}
```

Output-

Ch holds : X

New Value of Ch : Y

New Value of Ch : A

Boolean - The boolean type represents true/false values. Java defines the values true and false using the reserved words true and false. Thus, a variable or expression of type boolean will be one of these two values.

```
public class booltypeexp
{
    public static void main(String[] args)
    {
        boolean vl;
        vl = false;
        System.out.println("vl is : " + vl);
        vl = true;
        System.out.println("vl is : " + vl);
    }
}
```

Output-

vl is : false

vl is : true

Variables -

Variables are name of place holder which holds or store values of any data type. It is a named memory location that can store a value.

Constant -

A data item is constant when its value cannot be changed while a program is running.

```
public class constdemo {

    public static void main(String[] args)
    {
        final double vl = 5.4;
        System.out.println("Value of vl is : " + vl);
    }

}
```

Output-

Value of vl is : 5.4

Camel Casting -

Beginning an identifier with a lowercase letter and capitalizing subsequent words within the identifier is a style known as camel casing.

MITRC

LECTURE - 5

Casting, Automatic type promotion in expressions

Casting - It is a way to convert one data type to other by assigning only.

In java, there are two types of casting -

- **Widening Casting** - Widening casting is done automatically when passing a smaller size type to a larger size type - byte -> short -> char -> int -> long -> float -> double. It is also known as Automatic Type Conversion.

- **Narrowing Casting** - Narrowing casting must be done manually by placing the type in parentheses in front of the value from a larger type to a smaller size type - double -> float -> long -> int -> char -> short -> byte.

```
public class castingdemo {

    public static void main(String[] args) {

        //Widening Casting
        int int1 = 5;
        double dbl1 = int1;

        System.out.println(int1);
        System.out.println(dbl1);

        //Narrowing Casting
        double dbl2 = 3.45d;
        int int2 = (int) dbl2;

        System.out.println(dbl2);
        System.out.println(int2);
      }

}
```

Output-

5

5.0

3.45

3

Automatic type promotion in expression -

```
public class typepromoinexpresdemo {

    public static void main(String[] args) {
        byte a = 40;
        byte b = 50;
        byte c = 100;
        int d = a * b / c;
        System.out.println("Value of d is : " + d);
    }

}
```

Output-

Value of d is : 20

MITRC

LECTURE - 6

Operators, Operator Precedence, The ? Operator

Java provides a rich operator environment. An operator is a symbol that tells the compiler to perform a specific mathematical or logical manipulation. Java has four general classes of operators: arithmetic, bitwise, relational, and logical. Java also defines some additional operators that handle certain special situations.

Arithmetic Operators -

Java defines the following arithmetic operators:

Operator	Meaning
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
– –	Decrement

Relational and Logical Operators -

In the terms relational operator and logical operator, relational refers to the relationships that values can have with one another, and logical refers to the ways in which true and false values can be connected. Since the relational operators produce true or false results, they often work with the logical operators.

The relational operators are shown here:

Operator	Meaning
= =	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

The logical operators are shown here:

Operator	Meaning
&	AND
|	OR
^	XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	NOT

Assignment Operators -

The assignment operator is the single equal sign, =. This operator works in Java much as it does in any other computer language. It also allows us to create a chain of assignments. It has this general form:

var = expression;

var = value/variable = value/variable = value/variable ;

THE Shorthand Assignment Operators -

Java provides special shorthand assignment operators that simplify the coding of certain assignment statements.

+=	-=	*=	/=

Operator Precedence -

Operator precedence specifies the way operands are grouped with operators. For example, 1 + 2 * 3 is treated as 1 + (2 * 3), whereas 1 * 2 + 3 is treated as (1 * 2) + 3 because the multiplication operator has a higher precedence than the addition operator.

Highest						
++ (postfix)	– – (postfix)					
++ (prefix)	– – (prefix)	~	!	+ (unary)	– (unary)	(*type-cast*)
*	/	%				
+	–					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
|						
&&						
||						
?:						
–>						
=	op=					
Lowest						

Ternary Operator ?: -

Ternary operator ?: is used as one line replacement for if-then-else. It is the only conditional operator which takes three operands. It has this general form:

Var = logical check ? true part : false part;

LECTURE - 7

Arrays

An array is a named list of data items that all have the same data type. Each data item is an element of the array. We declare an array variable in the same way as we declare any simple variable, but we insert a pair of square brackets after the type.

The principal advantage of an array is that it organizes data in such a way that it can be easily manipulated. Although arrays in Java can be used just like arrays in other programming languages, they have one special attribute: they are implemented as objects.

One-Dimensional Arrays -

A one-dimensional array is a list of related variables. Such lists are common in programming. To declare a one-dimensional array, we use the following syntax-

type arrayname[] = new type[size];

The type declares the element type of the array. Arrays in Java are dynamically allocated using the new operator. The number of elements that the array will hold is determined by size.

Example -

```
public class odarraydemo1 {

    public static void main(String[] args) {
        int i;
        int vl[] = new int[10];
        for(i=0; i<10; i++)
        {
            vl[i] = i+1;
        }
        for(i=0;i<10;i++)
        {
            System.out.println("Value of vl at " + i + " is : " + vl[i]);
        }
    }
}
```

Output -

Value of vl at 0 is : 1

Value of vl at 1 is : 2

Value of vl at 2 is : 3

Value of vl at 3 is : 4

Value of vl at 4 is : 5

Value of vl at 5 is : 6

Value of vl at 6 is : 7

Value of vl at 7 is : 8

Value of vl at 8 is : 9

Value of vl at 9 is : 10

Arrays can also be initialized when they are created. The general form for initializing a one-dimensional array is:

type arrayname[] = { val1, val2, val3, ... , valN };

Example-

```
public class odarraydemo2 {

    public static void main(String[] args) {
        int i;
        int vl[] = {2,3,4,5,6,7,8,9,3,4};

        for(i=0;i<10;i++)
        {
            System.out.println("Value of vl at " + i + " is : " + vl[i]);
        }
    }
}
```

Output -

Value of vl at 0 is : 2

Value of vl at 1 is : 3

Value of vl at 2 is : 4

Value of vl at 3 is : 5

Value of vl at 4 is : 6

Value of vl at 5 is : 7

Value of vl at 6 is : 8

Value of vl at 7 is : 9

Value of vl at 8 is : 3

Value of vl at 9 is : 4

Multidimensional Arrays-

A multidimensional array is an array of arrays.

Two-Dimensional Arrays-

The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array contains one or more rows and one or more columns for each row. To declare a two-dimensional array, we use the following syntax-

type arrayname[][] = new type[size][size];

Arrays of Three or More Dimensions-

Java allows arrays with more than two dimensions. The general form of a multidimensional array declaration is:

type name[ ][ ]...[ ] = new type[size1][size2]...[sizeN];

MITRC

LECTURE - 8

Control Statements

Just like other language, Java supports various control statements as conditional statements, looping statements.

Conditional Statements-

If…else Statement-

In Java if...else statement works much like the IF...Else statement in any other language. It determines the flow of program execution based on whether some condition is true or false. Its simplest form is shown here:

if(condition) statement;

Example-

```
public class ifelsedemo {
    public static void main(String[] args) {
        int a=5;
        int b=6;
        if(a>b)
        {
            System.out.println("A is grater then B");
        }
        else
        {
            System.out.println("B is grater then or equal to A");
        }
    }
}
```

Output-

B is grater then or equal to A

Switch Statement-

The switch statement is useful when you need to test a single variable against a series of exact integer (including int, byte, and short types), character, or string values. The general form of switch statement is-

switch(expression)

```
{
        case value-1:
                case body;
                break;
        case value-2:
                case body;
                break;
        default:
                default body;
}
```

Example-

```
public class switchcasedemo {

    public static void main(String[] args) {
        int a=5;
        switch(a)
        {
            case 1:
                System.out.println(a);
                break;
            case 2:
                System.out.println(a);
                break;
            case 3:
                System.out.println(a);
                break;
            case 4:
                System.out.println(a);
                break;
            default:
                System.out.println(a);
        }
    }
}
```

Output-

5

Loops-

A loop is a structure that allows repeated execution of a block of statements. Within a looping structure, a Boolean expression is evaluated. If it is true, a block of statements called the loop body executes and the Boolean expression is evaluated again.

In Java, we can use several mechanisms to create loops.

- A while loop, in which the loop controlling Boolean expression is the first statement in the loop.
- A for loop, which is usually used as a concise format in which to execute loops.
- A do…while loop, in which the loop controlling Boolean expression is the last statement in the loop.

While Loop-

We can use a while loop to execute a body of statements continually as long as the Boolean expression that controls entry into the loop continues to be true. A while loop consists of the keyword while followed by a Boolean expression within parentheses, followed by the body of the loop. The general form to use a while loop is -

while(condition)

{

loop body;

}

Example-

```
public class whileloopexp {

    public static void main(String[] args) {
        int i=1;
        int a=5;
        int b=6;
        while(i==1)
        {
            int c = b % a;
            System.out.println("Remider is : " + c);
            b++;
            if(c==0)
            {
                i=c;
            }
        }
```


MITRC

```
        System.out.println("Reminder is Zero");
    }

}
```

Output-

Remider is : 1

Remider is : 2

Remider is : 3

Remider is : 4

Remider is : 0

Reminder is Zero

For Loop-

A for loop is a special loop that is used when a definite number of loop iterations is required. It provides a convenient way to create a counter-controlled loop. The general form to use a while loop is -

for(initialization; condition; iteration)

{

statement sequence

}

Example-1-

```
public class forloopexp {

    public static void main(String[] args) {
        for(int i=1;i<=10;i++)
        {
            int j = 2;
            int k = i * j;
            System.out.println(j + " X " + i + " = " + k);
        }
    }

}
```

MITRC

Output-

2 X 1 = 2

2 X 2 = 4

2 X 3 = 6

2 X 4 = 8

2 X 5 = 10

2 X 6 = 12

2 X 7 = 14

2 X 8 = 16

2 X 9 = 18

2 X 10 = 20

Example-2-

```
public class hello {

    public static void main(String[] args) {
        int i, j;
        for(i=0, j=10; i<j;i++,j--)
        {
            System.out.println("i and j : " + i + " " + j);
        }
    }
}
```

Output-

i and j : 0 10

i and j : 1 9

i and j : 2 8

i and j : 3 7

i and j : 4 6

do…while Loop-

the do…while loop checks its condition at the bottom of the loop. This means that a do…while loop will always execute at least once. The general form of the do…while loop is-

```
do {
        loop body;
} while(condition);
```

Example-

```
public class dowhileexp {

    public static void main(String[] args) {
        int i=1;
        int a=5;
        int b=6;
        do
        {
            int c = b % a;
            System.out.println("Remider is : " + c);
            b++;
            if(c==0)
            {
                i=c;
            }
        }while(i==1);
        System.out.println("Reminder is Zero");
    }
}
```

Output-

```
Remider is : 1
Remider is : 2
Remider is : 3
Remider is : 4
Remider is : 0
Reminder is Zero
```